

贪心算法答题模板

1 贪心算法基础

1.1 核心思想

每一步选择局部最优解，希望通过一系列局部最优选择达到全局最优。

关键：证明贪心选择性质和最优子结构。

1.2 适用条件

1. 贪心选择性质：局部最优选择能导致全局最优解
 2. 最优子结构：问题的最优解包含子问题的最优解
- ### 1.3 与动态规划的区别
- DP：考虑所有选择，自底向上，重叠子问题
 - 贪心：每次选局部最优，自顶向下，通常不回溯

2 区间调度问题

2.1 活动选择问题

问题：选最多互不相容的活动。

贪心策略：每次选结束时间最早的活动。

证明思路：交换论证法——用贪心选择的解替换任意最优解中的第一个活动。

```
1 def activity_selection(activities):
2     # 按结束时间排序
3     activities.sort(key=lambda x: x[1])
4     count = 0
5     last_end = -inf
6     for start, end in activities:
7         if start >= last_end:
8             count += 1
9             last_end = end
10    return count
```

2.2 区间覆盖问题

问题：用最少的区间覆盖目标区间 $[L, R]$ 。

贪心策略：每次选择能覆盖当前起点且右端点最远的区间。

复杂度： $O(n \log n)$ 排序 + $O(n)$ 扫描。

```
1 def interval_cover(intervals, L, R):
2     intervals.sort() # 按左端点排序
3     count = 0
4     cur_pos = L
5     i = 0
6     while cur_pos < R:
7         max_right = cur_pos
8         while i < len(intervals) and intervals[i][0] <=
9             cur_pos:
10                max_right = max(max_right, intervals[i][1])
11                i += 1
12            if max_right == cur_pos: # 无法覆盖
13                return -1
14            count += 1
15            cur_pos = max_right
16    return count
```

3 背包问题

3.1 分数背包（可分割物品）

问题：物品可分割，求最大价值。

贪心策略：按性价比（价值/重量）从高到低选择。

注意：与 0-1 背包不同，分数背包可用贪心。

```
1 def fractional_knapsack(weights, values, capacity):
2     # 按性价比排序
3     items = sorted(zip(weights, values),
4                     key=lambda x: x[1]/x[0], reverse=
5                     True)
6     total_value = 0
7     for w, v in items:
8         if capacity <= 0: break
9         take = min(w, capacity)
10        total_value += take * (v / w)
11        capacity -= take
12    return total_value
```

4 图论问题

4.1 最小生成树（MST）

Prim 算法：

思想：从任意点出发，每次添加距离当前树最近的点。

复杂度： $O((V + E) \log V)$ （优先队列优化）。

适用：稠密图。

```
1 def prim(graph, start):
2     visited = set()
3     min_heap = [(0, start)] # (distance, node)
4     total_weight = 0
5     while min_heap:
6         weight, node = heapq.heappop(min_heap)
7         if node in visited: continue
8         visited.add(node)
9         total_weight += weight
10        for nei, w in graph[node]:
11            if nei not in visited:
12                heapq.heappush(min_heap, (w, nei))
13    return total_weight
```

Kruskal 算法：

思想：按边权排序，依次添加不形成环的边。

关键：并查集判断连通性。

复杂度： $O(E \log E)$ 。

适用：稀疏图。

```
1 def kruskal(edges, n):
2     edges.sort() # 按边权排序
3     uf = UnionFind(n)
4     total_weight = 0
5     count = 0
6     for w, u, v in edges:
7         if uf.find(u) != uf.find(v):
8             uf.union(u, v)
9             total_weight += w
10            count += 1
11            if count == n - 1: break
12    return total_weight
```

4.2 最短路径

Dijkstra 算法:

问题: 单源最短路径 (非负权边)。

贪心策略: 每次选择距离源点最近的未访问点。

复杂度: $O((V + E) \log V)$ 。

注意: 不能处理负权边 (使用 Bellman-Ford)。

```
1 def dijkstra(graph, start):
2     dist = {node: inf for node in graph}
3     dist[start] = 0
4     visited = set()
5     min_heap = [(0, start)]
6     while min_heap:
7         d, node = heapq.heappop(min_heap)
8         if node in visited: continue
9         visited.add(node)
10        for nei, w in graph[node]:
11            if dist[node] + w < dist[nei]:
12                dist[nei] = dist[node] + w
13                heapq.heappush(min_heap, (dist[nei],
14                                         nei))
15    return dist
```

5 编码问题

5.1 哈夫曼编码

问题: 给定字符频率, 构造最短前缀编码。

贪心策略: 每次选择频率最小的两个节点合并。

数据结构: 最小堆 (优先队列)。

复杂度: $O(n \log n)$ 。

性质: 无前缀码, 频率高的字符编码短。

```
1 def huffman_encoding(freq):
2     heap = [(f, char) for char, f in freq.items()]
3     heapq.heapify(heap)
4     while len(heap) > 1:
5         f1, c1 = heapq.heappop(heap)
6         f2, c2 = heapq.heappop(heap)
7         heapq.heappush(heap, (f1 + f2, (c1, c2)))
8     return heap[0] # 哈夫曼树
```

6 经典贪心问题

6.1 跳跃游戏

问题: 从起点能否跳到终点。

贪心策略: 维护能到达的最远位置。

复杂度: $O(n)$ 。

```
1 def can_jump(nums):
2     max_reach = 0
3     for i, jump in enumerate(nums):
4         if i > max_reach: # 无法到达当前位置
5             return False
6         max_reach = max(max_reach, i + jump)
7     return True
```

6.2 加油站问题

问题: 环形路线, 选择起点使得能绕一圈。

贪心策略:

1. 总油量 < 总消耗则无解
2. 从剩余油量最低的站之后出发

```
1 def can_complete_circuit(gas, cost):
2     total_surplus = 0
3     surplus = 0
4     start = 0
5     for i in range(len(gas)):
6         total_surplus += gas[i] - cost[i]
7         surplus += gas[i] - cost[i]
8         if surplus < 0:
9             start = i + 1
10            surplus = 0
11    return start if total_surplus >= 0 else -1
```

6.3 分发饼干

问题: 每个孩子一块饼干, 满足最多孩子。

贪心策略: 小饼干分给胃口小的孩子。

排序后双指针匹配。

```
1 def find_content_children(greed, size):
2     greed.sort()
3     size.sort()
4     i = j = count = 0
5     while i < len(greed) and j < len(size):
6         if size[j] >= greed[i]:
7             count += 1
8             i += 1
9             j += 1
10        else:
11            j += 1
12    return count
```

7 贪心策略总结

7.1 常见贪心策略

1. 按时间排序: 活动选择 (结束时间)、区间调度
2. 按权值排序: 分数背包 (性价比)、哈夫曼编码 (频率)
3. 最短/最长: Dijkstra (最短路径)、Prim (最近点)
4. 区间覆盖: 每次选右端点最远

7.2 证明方法

1. 交换论证法: 证明贪心解可以替换任意最优解中的选择
2. 数学归纳法: 证明每一步选择都保持最优性
3. 反证法: 假设存在更优解, 导出矛盾

7.3 贪心 vs DP

- 0-1 背包: 必须用 DP (状态不可逆)
- 分数背包: 可用贪心 (可分割)
- 最短路径: Dijkstra (贪心 + DP 思想)
- 区间调度: 贪心足够